第 6 章 树

1. 树的概念

（可以在 ppt 中演示树的定义以及相关术语）

1.1 树的定义

树型结构是一类重要的非线性数据结构。

- 有一个特殊的结点，称为**根结点**，根结点没有前驱结点。
- 除根结点外，其余结点被分成 M 个互不相交的集合 T_1、T_2、...、T_m，其中每一个集合 T 又是一棵树，称这棵树为根节点的子树。

因此，树是**递归**定义的。

1.2 树的基本术语

- **结点的度**：树中一个结点孩子的个数称为该结点的度。
- **树的度：**树中结点最大的度数称为树的度。
- **树的高度（深度）**：树中结点的最大层数称为树的高度（深度）。
- **路径**：树中两个结点之间的路径是由这两个结点之间所经过的结点序列构成的，路径长度为序列中边的个数。

（可以在 ppt 中演示树在算法竞赛中的常见形式）

1.3 有序树和无序树

- 有序树：结点的子树按照从左往右的顺序排列，不能更改。
- 无序树：结点的子树之间没有顺序，随意更改。

这个认知会在我们后续学习二叉树的时候用到，因为二叉树需要区分左右孩子。

1.4 有根树和无根树

- 有根树：树的根节点已知，是固定的。
- 无根树：树的根节点未知，谁都可以是根结点。

这个认知主要会影响我们对于树的存储。在存储树结构的时候，我们最重要的就是要存下逻辑关系。

> 如果是无根树，父子关系不明确，此时我们需要把所有的情况都存下来。比如 a 和 b 之间有一条边，我们不仅要存 a 有一个孩子 b，也要存 b 有一个孩子 a。
>
> 甚至有的时候，在有根树的题目里，也要这样存储。

2. 树的存储

树结构相对线性结构来说就比较复杂。存储时，既要保存值域，也要保存结点与结点之间的关系。实际中树有很多种存储方式：双亲表示法，孩子表示法、孩子双亲表示法以及孩子兄弟表示法等。

现阶段，我们只用掌握**孩子表示法**，学会用孩子表示法存储树，并且在此基础上遍历整棵树。后续会在并查集中学习**双亲表示法**。至于其它存储形式，算法竞赛阶段不要求掌握。

2.1 孩子表示法

（用 ppt 展示，效果更佳）

孩子表示法是将每个结点的孩子信息存储下来。

如果是在**无根树**中，父子关系不明确，我们会将与该结点相连的所有的点都存储下来。

2.2 实现方式一：用 vector 数组实现

（用 ppt 展示，效果更佳）

案例：

题目描述：

> 给定一棵树，该树一共有 n 个结点，编号分别是 $1 \sim n$。

输入描述：

> 第一行一个整数 n ，表示 n 个结点。
>
> 接下来 $n-1$ 行，每行两个整数 u, v ，表示 u 和 v 之间有一条边。

$vector$ 是可变长数组，如果只涉及尾插，效率还是可以的。而树结构这种一对多的关系，正好可以利用尾插，把所有的关系全部存起来。

- 因此，可以创建一个大小为 $n+1$ 的 $vector$ 数组 $edges[n+1]$。
- 其中 $edges[i]$ 里面就保存着 i 号结点所连接的结点。

代码实现：

```cpp
#include <iostream>
#include <vector>

using namespace std;

const int N = 1e5 + 10;

int n;
vector<int> edges[N]; // 存储树

int main()
{
    cin >> n;
    for(int i = 1; i < n; i++)
    {
        int a, b; cin >> a >> b;
        // a 和 b 之间有一条边
        edges[a].push_back(b);
        edges[b].push_back(a);
    }

    return 0;
}
```

2.3 实现方式二：链式前向星

（用 ppt 展示，效果更佳）

链式前向星的本质就是用数组来模拟链表。因此，一定要先把《链表和 list》章节中用数组模拟实现链表这个操作搞懂。

代码实现：

```cpp
#include <iostream>

using namespace std;

const int N = 1e5 + 10;

// 链式前向星
int h[N], e[N * 2], ne[N * 2], id;
int n;

// 其实就是把 b 头插到 a 所在的链表后面
```

```cpp
void add(int a, int b)
{
    id++;
    e[id] = b;
    ne[id] = h[a];
    h[a] = id;
}

int main()
{
    cin >> n;
    for(int i = 1; i < n; i++)
    {
        int a, b; cin >> a >> b;
        // a 和 b 之间有一条边
        add(a, b); add(b, a);
    }

    return 0;
}
```

2.4 总结

关于 vector 数组以及链式前向星：

- 前者由于用到了容器 vector，实际运行起来相比较于后者更耗时，因为 vector 是动态实现的。
- 但是在如今的算法竞赛中，一般不会无聊到卡这种常数时间。也就是 vector 虽然慢，但不会因此而超时。

那么，在做题的时候，选择一种自己喜欢的方式即可。

因为两种实现方式都挺简单，搞清楚原理之后也都很容易理解。

3. 树的遍历

树的遍历就是**不重不漏**的将树中所有的点都扫描一遍。

在之前学过的线性结构中，遍历就很容易，直接从前往后扫描一遍即可。但是在树形结构中，如果不按照一定的规则遍历，就会漏掉或者重复遍历一些结点。因此，在树形结构中，要按照一定规则去遍历。常用的遍历方式有两种，一种是**深度优先遍历**，另一种是**宽度优先遍历**。

> **注意：树的遍历是许多关于树的算法的基石，一定要重点掌握。**

3.1 深度优先遍历 - DFS

（用 ppt 展示，效果更佳）

深度优先遍历，英文缩写为 DFS，全称是 Depth First Search，中文名是深度优先搜索，是一种用于遍历或搜索树或图的算法。所谓深度优先，就是说每次都尝试向更深的节点走，也就是一条路走到黑。

具体流程：

> 1. 从根节点出发，依次遍历每一棵子树；
> 2. 遍历子树的时候，重复第一步。

因此，深度优先遍历是一种递归形式的遍历，可以用递归来实现。

案例：

题目描述：

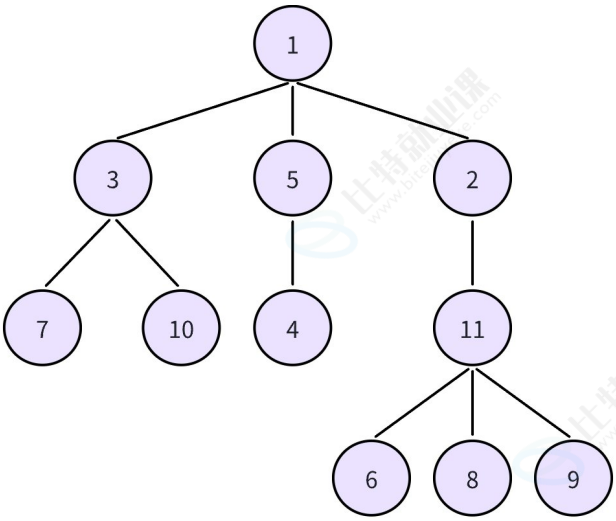
> 给定一棵树，该树一共有 n 个结点，编号分别是 $1 \sim n$。

输入描述：

> 第一行一个整数 n，表示 n 个结点。
>
> 接下来 $n-1$ 行，每行两个整数 u, v，表示 u 和 v 之间有一条边。

测试用例：

```
11
1 3
7 3
3 10
1 5
4 5
2 1
11 2
6 11
11 8
11 9
```



3.1.1 用 vector 数组存储

代码实现：

> 注：存储树结构的时候，会把相邻的所有结点都存下来，这样在扫描子树的时候会直接扫描到上一层，这不是我们想要的结果。
>
> 因此，需要一个 st 数组来标记，哪些结点已经访问过，接下来 dfs 的时候，就不去遍历那些点。

```
int n;
vector<int> edges[N]; // edges[i] 保存着 i 号结点相连的所有点
bool st[N]; // 标记当前结点是否已经被遍历过

// 当前遍历到 u 这棵子树
void dfs1(int u)
{
    // 先访问该点
    cout << u << " ";
    st[u] = true; // 标记该点已经被访问过

    // 访问它的子树
    for(auto v : edges[u])
    {
        if(!st[v]) dfs1(v); // 如果没有遍历过，再去遍历
    }
}

// 用 vector 数组
void test1()
{
    cin >> n;
```

```
23
24      for(int i = 1; i <= n - 1; i++)
25      {
26          int a, b; cin >> a >> b; // 读入一条边
27
28          edges[a].push_back(b); // 保存 a -> b 的一条边
29          edges[b].push_back(a); // 保存 b -> a 的一条边
30      }
31
32      dfs1(1);
33  }
```

3.1.2 链式前向星存储

代码实现：

```
1   int n;
2   int h[N], e[N * 2], ne[N * 2], id;
3   bool st[N]; // 标记当前结点是否已经被遍历过
4
5   void add(int a, int b)
6   {
7       id++;
8       e[id] = b; // 搞一个格子，存 b
9
10      // 把 b 头插在 a 这个链表的后面
11      ne[id] = h[a];
12      h[a] = id;
13  }
14
15  // 当前遍历到 u 这棵子树
16  void dfs2(int u)
17  {
18      cout << u << " ";
19      st[u] = true;
20
21      for(int i = h[u]; i; i = ne[i])
22      {
23          int v = e[i];
24          if(!st[v]) dfs2(v);
25      }
26  }
27
28  // 用数组模拟链表
```

```cpp
void test2()
{
    cin >> n;

    for(int i = 1; i <= n - 1; i++)
    {
        int a, b; cin >> a >> b;

        add(a, b); add(b, a);
    }

    dfs2(1);
}
```

时间复杂度：

> 简单估计一下，在 dfs 的整个过程中，会把树中所有的边扫描量两遍。边的数量为 $n-1$，因此时间复杂度为 $O(N)$。

空间复杂度：

> 最差情况下，结点个数为 n 的树，高度也是 n，也就是变成一个链表。此时递归的深度也是 n，此时的空间复杂度为 $O(N)$。

3.2 宽度优先遍历 - BFS

（用 ppt 展示，效果更佳）

宽度优先遍历，英文缩写为 BFS，全称是 Breadth First Search，也叫广度优先遍历。也是一种用于遍历或搜索树或图的算法。所谓宽度优先。就是每次都尝试访问同一层的节点。 如果同一层都访问完了，再访问下一层。

算法过程可以看做是在树和图中，在起点放上一个细菌，每秒向周围的那些干净的位置扩散一层，直到把所有位置都感染。

具体实现方式：借助队列。

1. 初始化一个队列；
2. 根节点入队，同时标记该节点已经入队；
3. 当队列不为空时，拿出队头元素，访问，然后将队头元素的所有孩子入队，同时打上标记；
4. 重复 3 过程，直到队列为空。

案例：

题目描述：

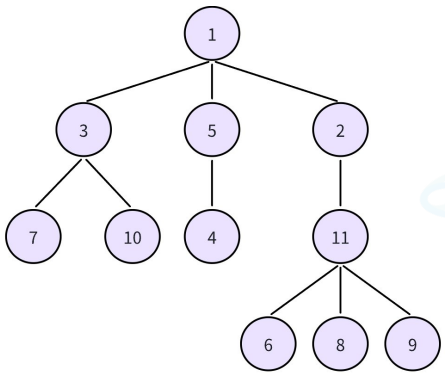
> 给定一棵树，该树一共有 n 个结点，编号分别是 $1 \sim n$。

输入描述：

> 第一行一个整数 n ，表示 n 个结点。
>
> 接下来 $n-1$ 行，每行两个整数 u, v ，表示 u 和 v 之间有一条边。

测试用例：

```
11
1 3
7 3
3 10
1 5
4 5
2 1
11 2
6 11
11 8
11 9
```



3.2.1 用 vector 数组存储

代码实现：

```
int n;
vector<int> edges[N]; // edges[i] 保存着 i 号结点相连的所有点
```

```cpp
bool st[N]; // 标记哪些点已经入过队了

void bfs1()
{
    queue<int> q;
    q.push(1);
    st[1] = true;

    while(q.size())
    {
        auto u = q.front(); q.pop();
        cout << u << " ";

        // 让孩子入队
        for(auto v : edges[u])
        {
            if(!st[v])
            {
                q.push(v);
                st[v] = true;
            }
        }
    }
}

// 用 vector 数组
void test1()
{
    cin >> n;

    for(int i = 1; i <= n - 1; i++)
    {
        int a, b; cin >> a >> b; // 读入一条边

        edges[a].push_back(b); // 保存 a -> b 的一条边
        edges[b].push_back(a); // 保存 b -> a 的一条边
    }

    bfs1();
}
```

3.2.2 链式向前星存储

代码实现：

```cpp
int n;
int h[N], e[N * 2], ne[N * 2], id;
bool st[N]; // 标记哪些点已经入过队了

void add(int a, int b)
{
    id++;
    e[id] = b; // 搞一个格子，存 b

    // 把 b 头插在 a 这个链表的后面
    ne[id] = h[a];
    h[a] = id;
}

void bfs2()
{
    queue<int> q;
    q.push(1);
    st[1] = true;

    while(q.size())
    {
        auto u = q.front(); q.pop();
        cout << u << " ";

        for(int i = h[u]; i; i = ne[i])
        {
            int v = e[i];
            if(!st[v])
            {
                q.push(v);
                st[v] = true;
            }
        }
    }
}

// 用数组模拟链表
void test2()
{
    cin >> n;

    for(int i = 1; i <= n - 1; i++)
    {
        int a, b; cin >> a >> b;

        add(a, b);
```

```
48              add(b, a);
49          }
50
51          bfs2();
52      }
```

时间复杂度：

> 所有结点只会入队一次，然后出队一次，因此时间复杂度为 $O(N)$。

空间复杂度：

> 最坏情况下，所有的非根结点都在同一层，此时队列里面最多有 $n-1$ 个元素，空间复杂度为 $O(N)$。

4. 树的算法题

> 这里不打算给大家准备树相关的算法题，因为涉及树的算法题都比较难，比如：搜索、剪枝、树形dp等等。这些内容，我们会在后面的算法篇章中细讲。
>
> 本章的重点还是**树的概念**以及**树的存储和遍历**上。大家只要把《树的存储》以及《树的遍历》中的代码好好写写，并且全部掌握就足矣。